

QUESTIONS

Q1

Carbon Aware Cloud
Computing Management
System
50 marks

Q2

Green Data Center Energy
Management System
50 marks

2/2 answered

Q2 Green Data Center Energy Management System

50 marks

A large organization operates a data center containing servers, cooling systems, and renewable-energy equipment and wants to reduce energy consumption and carbon emissions. Current issues include high server energy consumption, inefficient cooling, no distinction between renewable and non-renewable energy, difficulty adding new energy sources, and poor dynamic management. Develop a sustainable C++ OOP system using inheritance, multiple inheritance, virtual base classes, abstract classes, constructors and destructors, composition, pointers, this pointer, and runtime polymorphism. Design abstract DataCenterComponent with Server, CoolingUnit, and RenewableSource. Create GreenServer inheriting from Server and RenewableSource. Use virtual inheritance and an EnergyMeter member class. Implement pure virtual calculateEnergy(). Server uses CPU energy + memory energy; CoolingUnit uses cooling energy; RenewableSource calculates renewable energy generated. Demonstrate DataCenterComponent*, new, delete, overriding, this pointer, and a menu-driven program.

Your Code

```

1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 class EnergyMeter
6 {
7 private:
8     double energy;
9
10 public:
11     EnergyMeter(double e = 0)
12     {
13         energy = e;
14     }
15
16     void setEnergy(double e)
17     {
18         energy = e;
19     }
20
21     double getEnergy()
22     {
23         return energy;
24     }
25 };
26
27 class DataCenterComponent
28 {
29 protected:
30     int id;
31     string name;
32     EnergyMeter meter;
33
34 public:
35     DataCenterComponent(int i, string n, double e)
36         : meter(e)
37     {
38         this->id = i;
39         this->name = n;
40     }
41
42     virtual double calculateEnergy() = 0;
43
44     virtual void display()
45     {
46         cout << "ID: " << id << endl;
47         cout << "Name: " << name << endl;
48     }
49
50     virtual ~DataCenterComponent()

```

Input

stdin input

Output

Visible Test Cases

Test Case 1

```

51 {
52     cout << "Component destroyed" << endl;
53 }
54 };
55
56 class Server : virtual public DataCenterComponent
57 {
58 protected:
59     double cpuEnergy;
60     double memoryEnergy;
61
62 public:
63     Server(int i, string n, double e, double cpu, double memory)
64         : DataCenterComponent(i, n, e)
65     {
66         cpuEnergy = cpu;
67         memoryEnergy = memory;
68     }
69
70     double calculateEnergy() override
71     {
72         return cpuEnergy + memoryEnergy;
73     }
74
75     void display() override
76     {
77         cout << "\n===== SERVER =====< endr>
78         DataCenterComponent::display();
79         cout << "CPU Energy: " << cpuEnergy << " kWh" << endl;
80         cout << "Memory Energy: " << memoryEnergy << " kWh" << endl;
81         cout << "Total Energy: "
82             << calculateEnergy() << " kWh" << endl;
83     }
84 };
85
86 class CoolingUnit : virtual public DataCenterComponent
87 {
88 protected:
89     double coolingEnergy;
90
91 public:
92     CoolingUnit(int i, string n, double e, double cooling)
93         : DataCenterComponent(i, n, e)
94     {
95         coolingEnergy = cooling;
96     }
97
98     double calculateEnergy() override
99     {
100         return coolingEnergy;
101     }
102
103     void display() override
104     {
105         cout << "\n===== COOLING UNIT =====< endr>
106         DataCenterComponent::display();
107         cout << "Cooling Energy: "
108             << coolingEnergy << " kWh" << endl;
109         cout << "Total Energy: "
110             << calculateEnergy() << " kWh" << endl;
111     }
112 };
113
114 class RenewableSource : virtual public DataCenterComponent
115 {
116 protected:
117     double renewableEnergy;
118
119 public:
120     RenewableSource(int i, string n, double e, double renewable)
121         : DataCenterComponent(i, n, e)
122     {

```

```

128
129 double calculateEnergy() override
130 {
131     return renewableEnergy;
132 }
133
134 double getRenewableEnergy()
135 {
136     return renewableEnergy;
137 }
138
139 void display() override
140 {
141     cout << "\n===== RENEWABLE SOURCE =====> << endl;
142     DataCenterComponent::display();
143     cout << "Renewable Energy: "
144         << renewableEnergy << " kWh" << endl;
145     cout << "Generated Energy: "
146         << calculateEnergy() << " kWh" << endl;
147 }
148 };
149
150 class GreenServer : public Server, public RenewableSource
151 {
152 private:
153     string sourceType;
154
155 public:
156     GreenServer(int i, string n, double e,
157                 double cpu, double memory,
158                 double renewable, string source)
159         : DataCenterComponent(i, n, e),
160           Server(i, n, e, cpu, memory),
161           RenewableSource(i, n, e, renewable)
162     {
163         sourceType = source;
164     }
165
166 double calculateEnergy() override
167 {
168     double serverEnergy = Server::calculateEnergy();
169     double renewable = RenewableSource::calculateEnergy();
170
171     return serverEnergy - renewable;
172 }
173
174 void display() override
175 {
176     cout << "\n===== GREEN SERVER =====> << endl;
177
178     cout << "ID: " << id << endl;
179     cout << "Name: " << name << endl;
180     cout << "CPU Energy: "
181         << cpuEnergy << " kWh" << endl;
182     cout << "Memory Energy: "
183         << memoryEnergy << " kWh" << endl;
184     cout << "Server Energy: "
185         << Server::calculateEnergy() << " kWh" << endl;
186     cout << "Renewable Energy: "
187         << renewableEnergy << " kWh" << endl;
188     cout << "Renewable Source: "
189         << sourceType << endl;
190     cout << "Net Energy: "
191         << calculateEnergy() << " kWh" << endl;
192 }
193 };
194
195 void showMenu()
196 {
197     cout << "\n=====> << endl;
198     cout << "GREEN DATA CENTER ENERGY MANAGEMENT" << endl;

```

```

196     cout << "===== " << endl;
197     cout << "1. Server" << endl;
198     cout << "2. Cooling Unit" << endl;
199     cout << "3. Renewable Source" << endl;
200     cout << "4. Green Server" << endl;
201     cout << "5. Complete Energy Report" << endl;
202     cout << "6. Exit" << endl;
203 };
204
205 void runServer()
206 {
207     DataCenterComponent* ptr =
208         new Server(
209             101,
210             "Production Server",
211             0,
212             80,
213             40
214         );
215
216     ptr->display();
217
218     cout << "Energy through pointer: "
219         << ptr->calculateEnergy()
220         << " kWh" << endl;
221
222     delete ptr;
223 }
224
225 void runCooling()
226 {
227     DataCenterComponent* ptr =
228         new CoolingUnit(
229             102,
230             "Main Cooling Unit",
231             0,
232             60
233         );
234
235     ptr->display();
236
237     cout << "Energy through pointer: "
238         << ptr->calculateEnergy()
239         << " kWh" << endl;
240
241     delete ptr;
242 }
243
244 void runRenewable()
245 {
246     DataCenterComponent* ptr =
247         new RenewableSource(
248             103,
249             "Solar Power System",
250             0,
251             100
252         );
253
254     ptr->display();
255
256     cout << "Energy through pointer: "
257         << ptr->calculateEnergy()
258         << " kWh" << endl;
259
260     delete ptr;
261 }
262
263 void runGreenServer()
264 {
265     DataCenterComponent* ptr =
266         new GreenServer(
267             104,

```

```

267         100,
268         "Green Cloud Server",
269         0,
270         100,
271         50,
272         80,
273         "Solar Energy"
274     );
275
276     ptr->display();
277
278     cout << "Net Energy through pointer: "
279           << ptr->calculateEnergy()
280           << " kWh" << endl;
281
282     delete ptr;
283 }
284
285 void completeReport()
286 {
287     DataCenterComponent* components[4];
288
289     components[0] =
290         new Server(
291             201,
292             "Database Server",
293             0,
294             80,
295             40
296         );
297
298     components[1] =
299         new CoolingUnit(
300             202,
301             "Data Center Cooling",
302             0,
303             60
304         );
305
306     components[2] =
307         new RenewableSource(
308             203,
309             "Solar Power",
310             0,
311             100
312         );
313
314     components[3] =
315         new GreenServer(
316             204,
317             "Sustainable Server",
318             0,
319             100,
320             30,
321             80,
322             "Solar Energy"
323         );
324
325     double totalEnergy = 0;
326
327     cout << "\n===== " << endl;
328     cout << "COMPLETE ENERGY REPORT" << endl;
329     cout << "===== " << endl;
330
331     for (int i = 0; i < 4; i++)
332     {
333         components[i]->display();
334
335         totalEnergy =
336             totalEnergy +
337             components[i]->calculateEnergy();
338     }

```

```

339
340     cout << "\n===== " << endl;
341     cout << "TOTAL ENERGY: "
342         << totalEnergy << " kWh" << endl;
343     cout << "===== " << endl;
344
345     for (int i = 0; i < 4; i++)
346     {
347         delete components[i];
348     }
349 }
350
351 int main()
352 {
353     showMenu();
354
355     cout << "\nRunning all demonstrations automatically..." << endl;
356
357     runServer();
358     runCooling();
359     runRenewable();
360     runGreenServer();
361     completeReport();
362
363     cout << "\n===== " << endl;
364     cout << "PROGRAM COMPLETED SUCCESSFULLY" << endl;
365     cout << "===== " << endl;
366
367     return 0;
368 }

```

CO4_AT2_CASE_STUDY

View Only

← Back

QUESTIONS

Q1
Carbon Aware Cloud
Computing Management
System
50 marks**Q2**
Green Data Center Energy
Management System
50 marks

2/2 answered

Q1 Carbon-Aware Cloud Computing Management System

50 marks

A cloud service provider is modernizing its Carbon-Aware Cloud Management System to reduce the environmental impact of computing workloads. The system manages AI Workloads, Web Applications, and Data Analytics Workloads. Current issues include high energy consumption, inflexible energy-management strategies, difficulty supporting new workload categories, inefficient resource management, and inflexible carbon-emission calculation. Develop a C++ OOP application using inheritance, virtual base classes, abstract classes, constructors in derived classes, composition, pointers, the this pointer, pointers to derived classes, and runtime polymorphism. Design an abstract base class Workload with AIWorkload, WebWorkload, and AnalyticsWorkload. Create CriticalWorkload using multiple inheritance from AIWorkload and AnalyticsWorkload. Use virtual base classes to resolve ambiguity, and include an EnergyMonitor member class. Implement pure virtual calculateCarbon(). Demonstrate Workload*, new, delete, this pointer, pointer to derived class, and a menu-driven program.

Your Code

```
1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 class EnergyMonitor
6 {
7     double energy, intensity;
8
9 public:
10     EnergyMonitor(double e = 0, double i = 0.4)
11     {
12         energy = e;
13         intensity = i;
14     }
15
16     double calculateCarbon()
17     {
18         return energy + intensity;
19     }
20
21     void display()
22     {
23         cout << "Energy: " << energy << " kWh" << endl;
24         cout << "Carbon Intensity: " << intensity << " kg CO2/kWh" << endl;
25         cout << "Carbon Emission: " << calculateCarbon() << " kg CO2" << endl;
26     }
27 };
28
29 class Workload
30 {
31 protected:
32     int id;
33     string name;
34     double cpu, memory;
35     EnergyMonitor monitor;
36
37 public:
38     Workload(int i, string n, double c, double m, double e)
39     {
40         this->id = i;
41         this->name = n;
42         this->cpu = c;
43         this->memory = m;
44         monitor = EnergyMonitor(e);
45     }
46
47     virtual double calculateCarbon() = 0;
48
49     virtual void display()
50     {
```

Input

5

Output



Visible Test Cases

Test Case 1


```

51     cout << "ID: " << id << endl;
52     cout << "Name: " << name << endl;
53     cout << "CPU: " << cpu << " cores" << endl;
54     cout << "Memory: " << memory << " GB" << endl;
55     monitor.display();
56 }
57
58 virtual ~Workload() {}
59 };
60
61 class AIWorkload : virtual public Workload
62 {
63     double gpu;
64
65 public:
66     AIWorkload(int i, string n, double c, double m, double e, double g)
67         : Workload(i, n, c, m, e)
68     {
69         gpu = g;
70     }
71
72     double calculateCarbon() override
73     {
74         return monitor.calculateCarbon();
75     }
76
77     void display() override
78     {
79         cout << "\n--- AI WORKLOAD ---" << endl;
80         Workload::display();
81         cout << "GPU: " << gpu << " units" << endl;
82         cout << "Carbon: " << calculateCarbon() << " kg CO2" << endl;
83     }
84 };
85
86 class WebWorkload : public Workload
87 {
88     int users;
89
90 public:
91     WebWorkload(int i, string n, double c, double m, double e, int u)
92         : Workload(i, n, c, m, e)
93     {
94         users = u;
95     }
96
97     double calculateCarbon() override
98     {
99         return monitor.calculateCarbon();
100     }
101
102     void display() override
103     {
104         cout << "\n--- WEB WORKLOAD ---" << endl;
105         Workload::display();
106         cout << "Users: " << users << endl;
107         cout << "Carbon: " << calculateCarbon() << " kg CO2" << endl;
108     }
109 };
110
111 class AnalyticsWorkload : virtual public Workload
112 {
113     double data;
114
115 public:
116     AnalyticsWorkload(int i, string n, double c, double m, double e, d
117         : Workload(i, n, c, m, e)
118     {
119         data = d;
120     }
121
122     double calculateCarbon() override

```



```

123 {
124     return monitor.calculateCarbon();
125 }
126
127 void display() override
128 {
129     cout << "\n--- ANALYTICS WORKLOAD ---" << endl;
130     Workload::display();
131     cout << "Data Size: " << data << " GB" << endl;
132     cout << "Carbon: " << calculateCarbon() << " kg CO2" << endl;
133 }
134 },
135
136 class CriticalWorkload : public AIWorkload, public AnalyticsWorkload
137 {
138     string priority;
139
140 public:
141     CriticalWorkload(int i, string n, double c, double m,
142                     double s, double g, double d, string p)
143         : Workload(i, n, c, m, s),
144           AIWorkload(i, n, c, m, s, g),
145           AnalyticsWorkload(i, n, c, m, s, d)
146     {
147         priority = p;
148     }
149
150     double calculateCarbon() override
151     {
152         return monitor.calculateCarbon();
153     }
154
155     void display() override
156     {
157         cout << "\n--- CRITICAL WORKLOAD ---" << endl;
158         Workload::display();
159         cout << "GPU: " << gpuValue() << " units" << endl;
160         cout << "Data Size: " << dataValue() << " GB" << endl;
161         cout << "Priority: " << priority << endl;
162         cout << "Carbon: " << calculateCarbon() << " kg CO2" << endl;
163     }
164
165     double gpuValue()
166     {
167         return 5;
168     }
169
170     double dataValue()
171     {
172         return 1000;
173     }
174 },
175
176 int main()
177 {
178     Workload* w1 = new AIWorkload(
179         101, "AI Model Training", 16, 22, 50, 0);
180
181     Workload* w2 = new WebWorkload(
182         102, "E-Commerce Website", 8, 16, 20, 5000);
183
184     Workload* w3 = new AnalyticsWorkload(
185         103, "Business Analytics", 12, 24, 35, 500);
186
187     Workload* w4 = new CriticalWorkload(
188         104, "Critical AI Analytics", 24, 64, 60, 12, 1000, "HIGH");
189
190     cout << "CARBON-AWARE CLOUD MANAGEMENT SYSTEM" << endl;
191     cout << "===== " << endl;
192
193     w1->display();
194     w2->display();

```



```
191     cout << "===== " << endl;
192
193     w1->display();
194     w2->display();
195     w3->display();
196     w4->display();
197
198     double total;
199
200     total = w1->calculateCarbon()
201             + w2->calculateCarbon()
202             + w3->calculateCarbon()
203             + w4->calculateCarbon();
204
205     cout << "\n===== " << endl;
206     cout << "TOTAL CARBON EMISSION: "
207           << total << " kg CO2" << endl;
208     cout << "===== " << endl;
209
210     delete w1;
211     delete w2;
212     delete w3;
213     delete w4;
214
215     return 0;
216 }
```